

Project 9

Apache2 Server Deployment on Kubernetes — Access via Host Machine

23070122005 — Arya Balwadkar

Aim

To deploy an Apache2 (httpd) web server inside a Kubernetes Deployment, expose it using a NodePort Service, and access the running server from the host machine using Minikube commands.

PHASE 1 The Apache Deployment

Apache's official Docker image is named httpd. We will instruct Kubernetes to pull this image and keep a replica of it running in our cluster.

1 Verify the Cluster

Make sure Minikube is still awake and ready to accept commands:

POWERSHELL

```
minikube status
```

Ensure it says Running before proceeding.

2 Create the Deployment Manifest

Create a new file in your Project_9 folder named apache-deployment.yaml and paste the following architecture blueprint:

YAML — apache-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: apache-server-deployment
  labels:
    app: apache-web
spec:
  replicas: 1
  selector:
    matchLabels:
      app: apache-web
  template:
    metadata:
      labels:
        app: apache-web
```

```
spec:
  containers:
  - name: apache-container
    # httpd is the official Apache image on Docker Hub
    image: httpd:latest
    ports:
      # Apache runs on port 80 by default
      - containerPort: 80
```

```
PS F:\SEM_7\Devops_Lab\Project_9> minikube start --driver=docker
🐳 minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
🔧 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.50 ...
🔄 Restarting existing docker container for "minikube" ...
🌐 Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
🔍 Verifying Kubernetes components...
   ▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.1
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: default-storageclass, storage-provisioner, metrics-server
🎉 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS F:\SEM_7\Devops_Lab\Project_9> minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

3 Apply the Configuration

Run this command to send the blueprint to Kubernetes:

POWERSHELL

```
kubectl apply -f apache-deployment.yaml
```

```
PS F:\SEM_7\Devops_Lab\Project_9> kubectl apply -f apache-deployment.yaml
deployment.apps/apache-server-deployment created
```

4 Verify the Pod

Check if Kubernetes successfully pulled the Apache image and started the container:

POWERSHELL

```
kubectl get pods
```

```

● PS F:\SEM_7\Devops_Lab\Project_9> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
annotation-api-deployment-64596c6f44-6g272   1/1     Running   1 (4m35s ago)   3d19h
annotation-ui-deployment-9d44bc4f9-78hcr     1/1     Running   1 (4m36s ago)   3d19h
apache-server-deployment-69688cb64d-lspgw     1/1     Running   0             2m49s
auth-api-deployment-9c9bc4df-scsmn           1/1     Running   1 (4m36s ago)   3d20h
backend-deployment-66cb79749-26x7f           1/1     Running   10 (4m35s ago)  6d18h
backend-deployment-66cb79749-4dk9x           1/1     Running   1 (4m35s ago)   3d19h
frontend-deployment-74b9f7fc98-slvvs         1/1     Running   7 (2m52s ago)   6d18h
mongo-express-deployment-645f678fc7-tw6s8    1/1     Running   2 (4m35s ago)   4d13h
mongodb-deployment-594d6f6c44-6w7fg          1/1     Running   3             4d15h
○ PS F:\SEM_7\Devops_Lab\Project_9>

```

PHASE 2 Exposing the Server (The Service)

In Kubernetes, we use a Service of type NodePort to open a specific port on the cluster's network and map it directly to port 80 on your Apache container.

1 Create the Service Manifest

In your Project_9 folder, create a new file named apache-service.yaml and paste this configuration:

YAML — apache-service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: apache-nodeport-service
spec:
  # NodePort tells Kubernetes to expose this on the external network
  type: NodePort
  selector:
    # This must match the label we gave to the Apache pod in Phase 1
    app: apache-web
  ports:
    - port: 80
      targetPort: 80

```

2 Apply the Configuration

Run this command to build the network bridge:

POWERSHELL

```
kubectl apply -f apache-service.yaml
```

3 Verify the Network

Let's check if Kubernetes successfully created the service and assigned it a port:

POWERSHELL

```
kubectl get svc
```

Look for apache-nodeport-service in the list. Under the TYPE column, it should say NodePort. Under the PORT(S) column, you will see 80: followed by a random 5-digit number (like 31456). That 5-digit number is the specific door opened for your laptop.

```
PS F:\SEM_7\Devops_Lab\Project_9> kubectl apply -f apache-service.yaml
service/apache-nodeport-service created
PS F:\SEM_7\Devops_Lab\Project_9> kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
annotation-api-service	NodePort	10.109.232.36	<none>	5002:32447/TCP	3d19h
annotation-ui-service	NodePort	10.102.186.197	<none>	3000:31464/TCP	3d19h
apache-nodeport-service	NodePort	10.97.228.121	<none>	80:32014/TCP	6s
auth-api-service	NodePort	10.98.146.128	<none>	5001:30299/TCP	3d20h
backend-service	ClusterIP	10.98.215.229	<none>	5000/TCP	6d18h
frontend-service	NodePort	10.106.146.65	<none>	80:30080/TCP	6d18h
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	6d18h
mongo-express-service	NodePort	10.106.3.131	<none>	8081:31705/TCP	4d13h
mongodb-service	ClusterIP	10.100.88.19	<none>	27017/TCP	4d15h

```
PS F:\SEM_7\Devops_Lab\Project_9>
```

PHASE 3 Bridging the Network and Verifying Access

Because you are using Minikube with the Docker driver on Windows, you cannot just type localhost:31456 into your browser. We have to command Minikube to create a live tunnel that connects your computer's network directly to the Kubernetes NodePort.

1 Open the Tunnel

Run this command in your PowerShell terminal:

POWERSHELL

```
minikube service apache-nodeport-service
```

2 Keep the Terminal Open

Just like we saw with the Mongo Express and React UI projects, Minikube will output a small table with a local URL (e.g., http://127.0.0.1:XXXXX) and automatically attempt to open your default web browser.

Crucial reminder: Do not close this PowerShell window or press Ctrl+C, or the network tunnel will instantly collapse!

3 Verify the Webpage

If the browser opens successfully (or if you copy/paste the 127.0.0.1 URL into Chrome/Edge manually), you should be greeted by a very simple, iconic webpage.

By default, a fresh Apache container serves a blank white page with exactly three words in bold text: **"It works!"**

It works!

```
PS F:\SEM_7\Devops_Lab\Project_9> minikube service apache-nodeport-service
```

NAMESPACE	NAME	TARGET PORT	URL
default	apache-nodeport-service	80	http://192.168.49.2:32014

🔑 Starting tunnel for service apache-nodeport-service.

NAMESPACE	NAME	TARGET PORT	URL
default	apache-nodeport-service		http://127.0.0.1:63484

🌐 Opening service default/apache-nodeport-service in default browser...

! Because you are using a Docker driver on windows, the terminal needs to be open to run it.